PROJECT REPORT

Automated Linux Web Server Deployment

Using AWS EC2, Bash Scripting & Amazon S3

NAME : ATHARVA VADAGE

Domain	Cloud Computing / DevOps
Cloud Platform	Amazon Web Services (AWS)
Core AWS Services	EC2, S3, IAM, VPC, Security Groups
Technologies Used	Linux, Bash Scripting, Apache, HTML/CSS, AWS CLI
Region	Europe (Frankfurt) — eu-central-1
Date	September 2026

1. Abstract

This project demonstrates the design and implementation of an automated web server deployment pipeline on Amazon Web Services (AWS). A Linux-based EC2 instance was provisioned and configured to host a static website using the Apache HTTP Server. Website content (HTML, CSS, and JavaScript files) is stored in an Amazon S3 bucket, acting as a central content repository, and is synchronized to the web server using the AWS Command Line Interface (AWS CLI). A custom Bash script, deploy.sh, was developed to automate the end-to-end deployment process — including system updates, Apache installation and configuration, and content synchronization from S3 — reducing manual effort and ensuring a consistent, repeatable deployment. The project also covers the configuration of supporting AWS infrastructure such as IAM roles for secure programmatic access, a Virtual Private Cloud (VPC) for network isolation, and Security Groups to control inbound and outbound traffic.

2. Objectives

1. To provision and configure a Linux-based virtual server (EC2 instance) on AWS.
2. To install and configure the Apache HTTP Server to host a static website.
3. To use Amazon S3 as a centralized storage location for web content.
4. To automate deployment tasks using Bash scripting and the AWS CLI.
5. To secure the infrastructure using IAM roles/policies, VPC networking, and Security Groups.
6. To validate the deployment by accessing the hosted website via its public IP address.

3. AWS Services Used

Service	Purpose in the Project
Amazon EC2	Provides the virtual Linux server (compute instance) that runs Apache and hosts the website.
Amazon S3	Acts as a central object storage bucket holding the website's source files (HTML, CSS, JS), used as the single source of truth for deployments.
IAM	Manages secure, least-privilege access so the EC2 instance/CLI user can interact with S3 without hard-coding credentials.
VPC	Provides an isolated virtual network in which the EC2 instance is launched, including subnets and routing.
Security Groups	Acts as a virtual firewall controlling inbound (SSH - port 22, HTTP - port 80) and outbound traffic to the instance.

4. Technologies Used

Technology	Role
Linux (Amazon Linux)	Operating system running on the EC2 instance.
Bash Scripting	Automates package updates, Apache setup, and content deployment via deploy.sh.
Apache HTTP Server	Serves the static website content over HTTP on port 80.
HTML / CSS	Front-end content of the hosted website (index.html, style.css, script.js).
AWS CLI	Command-line tool used to interact with S3 (aws s3 sync, aws s3 ls) and other AWS services.

5. System Architecture

The architecture follows a simple but effective cloud deployment pattern: source website files are uploaded to an S3 bucket, which serves as the deployment repository. An EC2 instance, launched within a VPC and protected by a Security Group, runs the Apache web server. A Bash automation script executes on the instance to pull the latest content from S3 using the AWS CLI and restart/reload Apache as required, making the updated site immediately available over the internet via the instance's public IP address.

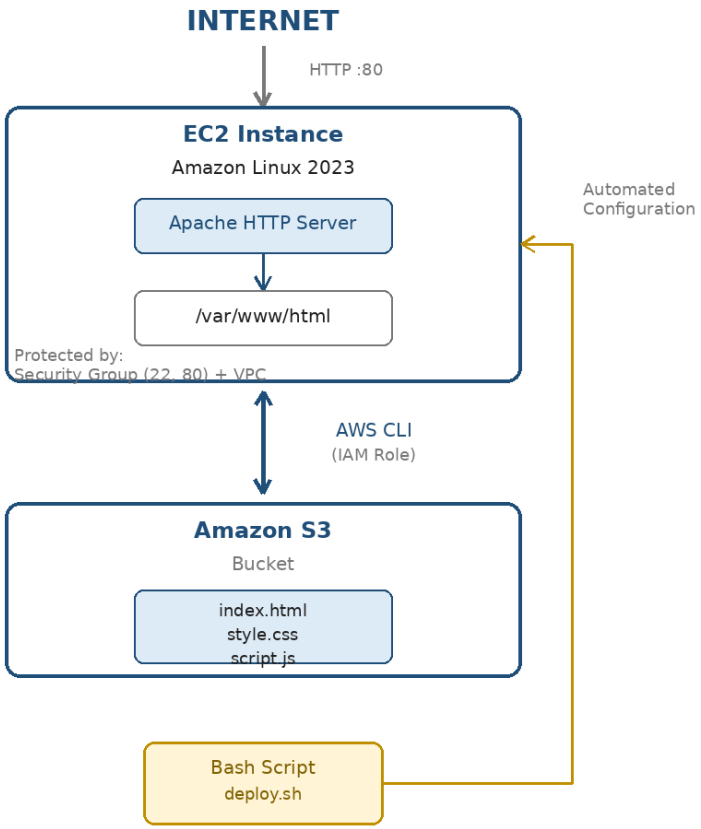


Figure 1: System Architecture Diagram

- User / Developer uploads website files to an S3 bucket.
- EC2 instance (within a VPC, secured by a Security Group) runs the deploy.sh script.
- deploy.sh updates system packages, ensures Apache is installed and running, and syncs content from S3 to /var/www/html using aws s3 sync.
- Apache serves the synced content to end users over HTTP (port 80).
- IAM permissions authorize the EC2 instance/CLI to read from the S3 bucket.

6. Implementation Steps

6.1 Launching the EC2 Instance

An Amazon Linux EC2 instance was launched in the eu-central-1 (Europe – Frankfurt) region within a custom VPC. A key pair was configured for secure access, and the instance was connected to using EC2 Instance Connect (browser-based SSH).

6.2 Configuring Security Groups

A Security Group was attached to the instance allowing inbound traffic on:

- Port 22 (SSH) — for remote administration and EC2 Instance Connect.
- Port 80 (HTTP) — to allow public access to the hosted website.

6.3 Setting Up IAM Permissions

An IAM role/policy was configured to grant the EC2 instance the necessary permissions to access the S3 bucket (e.g., s3:GetObject, s3:ListBucket), enabling the AWS CLI on the instance to run commands such as aws s3 ls and aws s3 sync without needing to embed long-term access keys on the instance.

6.4 Installing and Configuring Apache

The Apache HTTP Server (httpd) was installed on the EC2 instance and enabled as a system service so that it starts automatically on boot. Once started, the service status confirmed that Apache was configured and listening on port 80:

```
sudo yum install -y httpd
sudo systemctl enable httpd
sudo systemctl start httpd
sudo systemctl status httpd
```

The website content is served from Apache's default document root, /var/www/html, which contains index.html, style.css, and script.js.

6.5 Creating and Populating the S3 Bucket

An S3 bucket (e.g., atharva242005) was created to store the website's source files. The AWS CLI was used to verify bucket access and to upload/sync content:

```
aws s3 ls
sudo aws s3 sync s3://atharva242005 /var/www/html/
```

This command downloads the latest version of the website files from the S3 bucket directly into Apache's web root, ensuring the live site always reflects the content stored in S3.

6.6 Automating Deployment with a Bash Script

A Bash script, deploy.sh, was written to automate the full deployment workflow in a single command. The script was made executable and run with elevated privileges so that system-level operations (package updates, service management) succeed:

```
nano deploy.sh
chmod +x deploy.sh
sudo ./deploy.sh
```

The script performs the deployment in sequential stages, printing progress to the console, for example:

```
==================================
 AWS Web Server Deployment
==================================
[1/5] Updating packages...
[2/5] Installing Apache...
[3/5] Starting and enabling Apache service...
[4/5] Syncing website content from S3...
[5/5] Deployment complete. Logging results...
```

Deployment activity is logged to /var/log/webserver-deployment.log for auditing and troubleshooting purposes.

6.7 Verifying the Deployment

After deployment, the website was verified by navigating to the EC2 instance's public IP address in a web browser, confirming that Apache correctly served the synced HTML/CSS/JS content from S3.

7. Challenges Faced and Troubleshooting

Issue Encountered	Root Cause & Resolution
"bash: syntax error near unexpected token `newline'" when typing HTML directly into the terminal.	Raw HTML (e.g., ...) was mistakenly entered as a shell command. Bash interpreted the < character as input redirection. Resolved by writing HTML content into files using echo/tee or a text editor (nano) instead of typing it directly at the prompt.
"Permission denied" when writing to /var/www/html/index.html.	The file was owned by the apache user/group, while the logged-in ec2-user lacked write permission. Resolved by piping output through sudo tee -a instead of using plain shell redirection (>>), which does not inherit sudo privileges.
"Error: This command has to be run with superuser privileges" during deploy.sh execution.	The script's package-update step requires root privileges, but the script was initially executed as a normal user. Resolved by re-running the script with sudo ./deploy.sh.
"/var/log/webserver-deployment.log: No such file or directory".	A downstream effect of the previous error — the script exited before reaching the logging step, so the log file was never created. Resolved automatically once the script completed successfully after being run with sudo.

8. Results

The completed project delivers a fully functioning, repeatable deployment pipeline:

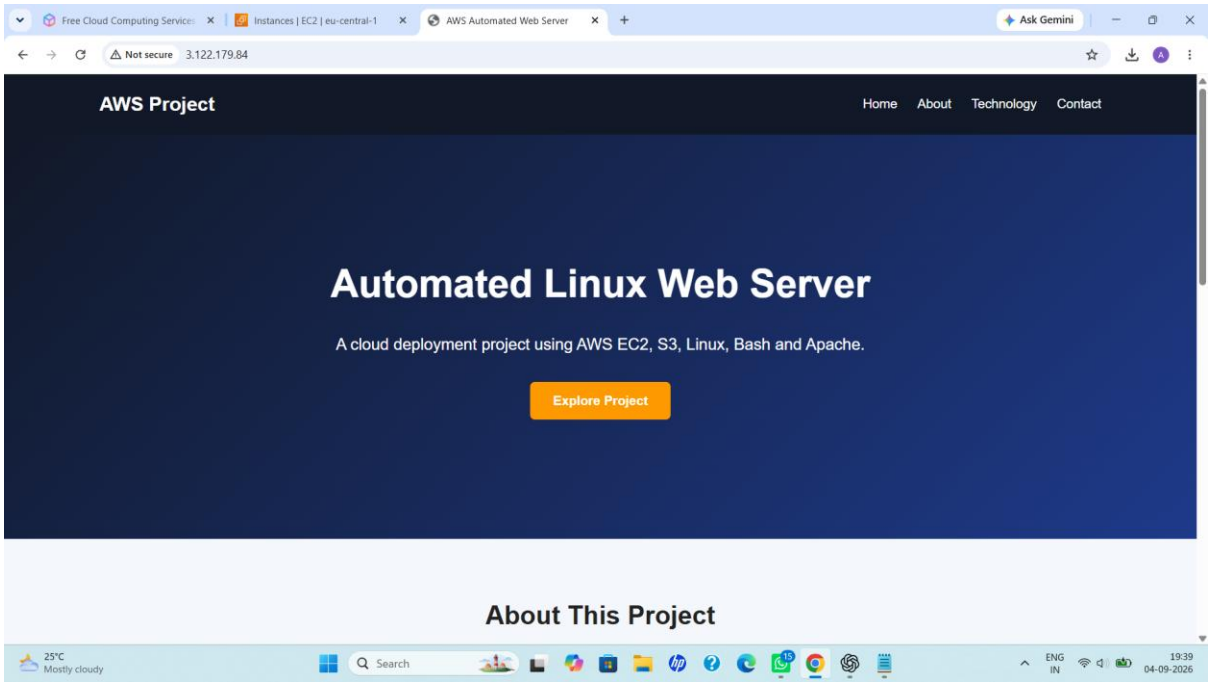
- A live Apache web server hosted on an AWS EC2 instance, accessible via its public IP address.
- Website content centrally managed and version-controlled through an S3 bucket.
- A single Bash script (deploy.sh) capable of deploying or redeploying the site end-to-end.
- Secure access enforced through IAM permissions, VPC networking, and Security Group rules.

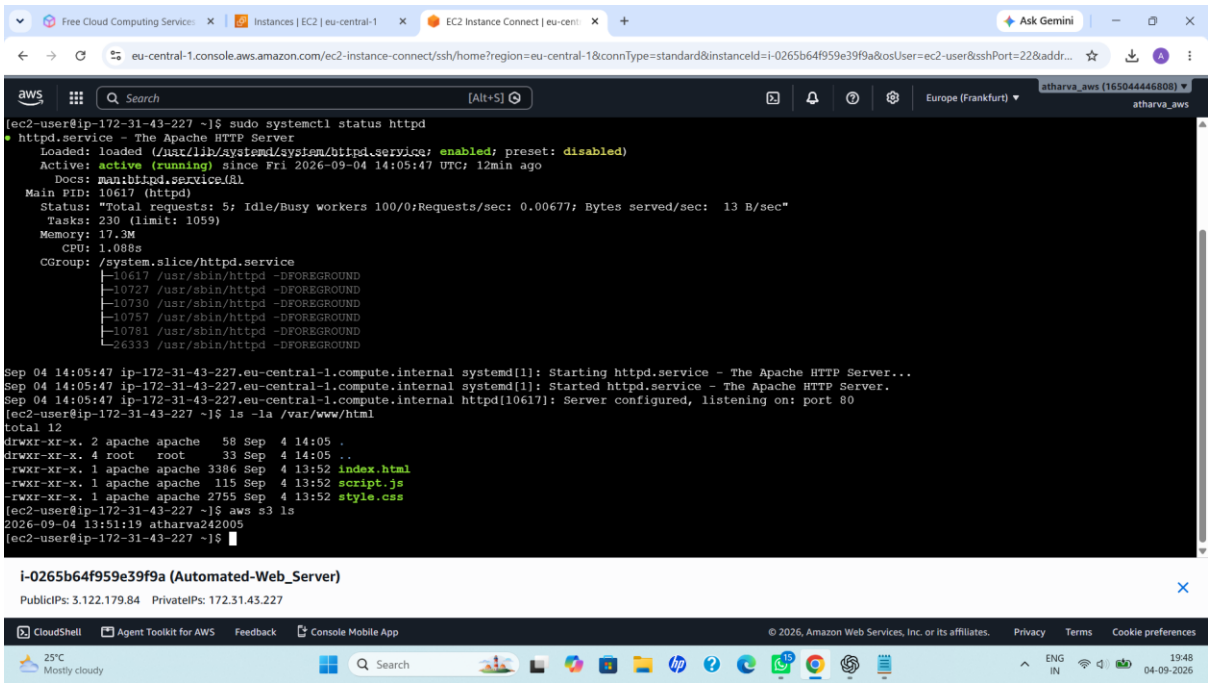
9. Conclusion

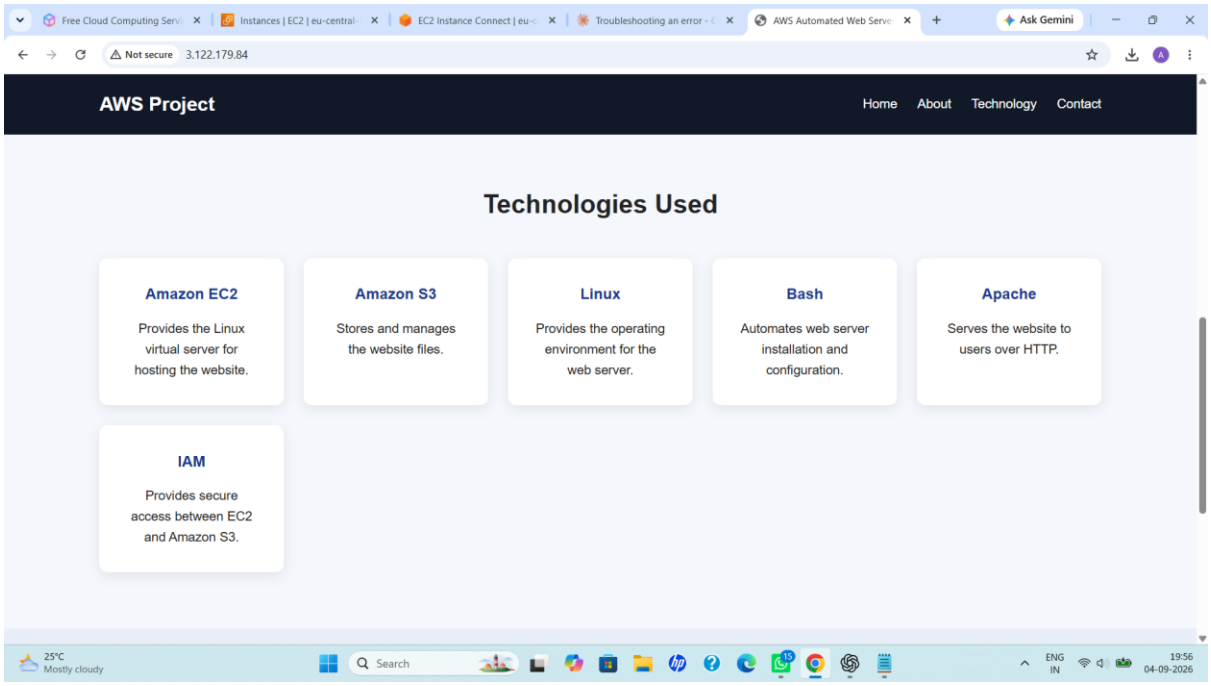
This project provided practical, hands-on experience in cloud infrastructure provisioning and DevOps-style automation using AWS. By combining EC2, S3, IAM, and Bash scripting, a reliable and repeatable deployment process was built for hosting a static website. The troubleshooting encountered along the way — particularly around Linux file permissions and privilege escalation with sudo — reinforced key operating system and cloud security concepts that are essential for real-world system administration and DevOps roles.

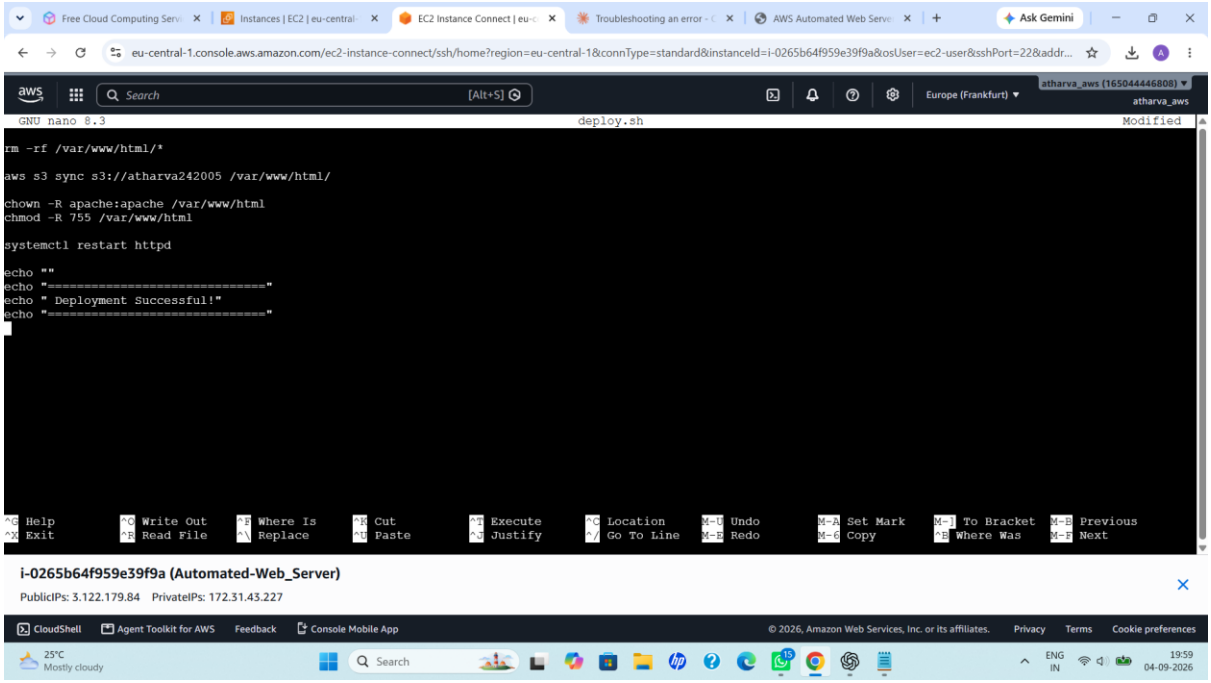
10. Future Scope

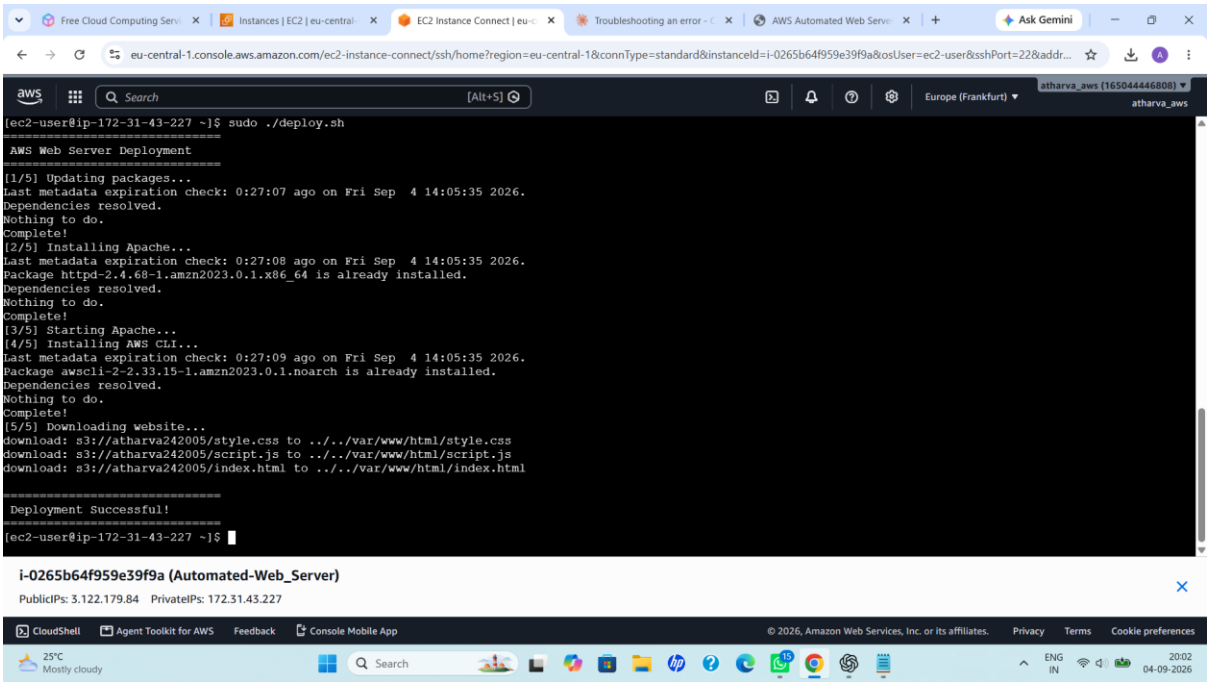
- Integrate an Application Load Balancer and Auto Scaling Group for high availability.
- Use AWS CodeDeploy or a CI/CD pipeline (e.g., GitHub Actions, Jenkins) to trigger deployments automatically on code changes.
- Enable HTTPS using AWS Certificate Manager and a Load Balancer or Let's Encrypt with Certbot.
- Store deployment logs in CloudWatch for centralized monitoring and alerting.
- Use Infrastructure as Code (e.g., Terraform or AWS CloudFormation) to provision the entire stack reproducibly.


Free Cloud Computing Services
Instances | EC2 | eu-central-1
AWS Automated Web Server
Ask Gemini
Not secure
3.122.179.84
AWS Project
Home
About
Technology
Contact
Automated Linux Web Server
A cloud deployment project using AWS EC2, S3, Linux, Bash and Apache.
Explore Project
About This Project
25°C
Mostly cloudy
Search
ENG
IN
19:39
04-09-2026


Free Cloud Computing Services
Instances | EC2 | eu-central-1
EC2 Instance Connect | eu-cent
Ask Gemini
eu-central-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=eu-central-1&connType=standard&instanceId=i-0265b64f959e39f9a&osUser=ec2-user&sshPort=22&addr...
aws
Search
[Alt+S]
Europe (Frankfurt)
atharva_aws (165044446808)
atharva_aws
[ec2-user@ip-172-31-43-227 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
Active: active (running) since Fri 2026-09-04 14:05:47 UTC; 12min ago
Docs: man:httpd.service(8)
Main PID: 10617 (httpd)
Status: "Total requests: 5; Idle/Busy workers 100/0;Requests/sec: 0.00677; Bytes served/sec: 13 B/sec"
Tasks: 230 (limit: 1059)
Memory: 17.3M
CPU: 1.088s
CGroup: /system.slice/httpd.service
├─10617 /usr/sbin/httpd -DFOREGROUND
├─10727 /usr/sbin/httpd -DFOREGROUND
├─10730 /usr/sbin/httpd -DFOREGROUND
├─10757 /usr/sbin/httpd -DFOREGROUND
├─10781 /usr/sbin/httpd -DFOREGROUND
└─26333 /usr/sbin/httpd -DFOREGROUND
Sep 04 14:05:47 ip-172-31-43-227.eu-central-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Sep 04 14:05:47 ip-172-31-43-227.eu-central-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Sep 04 14:05:47 ip-172-31-43-227.eu-central-1.compute.internal httpd[10617]: Server configured, listening on: port 80
[ec2-user@ip-172-31-43-227 ~]$ ls -la /var/www/html
total 12
drwxr-xr-x. 2 apache apache 58 Sep 4 14:05 .
drwxr-xr-x. 4 root root 33 Sep 4 14:05 ..
-rwxr-xr-x. 1 apache apache 3386 Sep 4 13:52 index.html
-rwxr-xr-x. 1 apache apache 115 Sep 4 13:52 script.js
-rwxr-xr-x. 1 apache apache 2755 Sep 4 13:52 style.css
[ec2-user@ip-172-31-43-227 ~]$ aws s3 ls
2026-09-04 13:51:19 atharva242005
[ec2-user@ip-172-31-43-227 ~]$
i-0265b64f959e39f9a (Automated-Web_Server)
PublicIPs: 3.122.179.84 PrivateIPs: 172.31.43.227
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
25°C
Mostly cloudy
Search
ENG
IN
19:48
04-09-2026


Not secure 3.122.179.84
AWS Project
Home
About
Technology
Contact
Technologies Used
Amazon EC2
Provides the Linux virtual server for hosting the website.
Amazon S3
Stores and manages the website files.
Linux
Provides the operating environment for the web server.
Bash
Automates web server installation and configuration.
Apache
Serves the website to users over HTTP.
IAM
Provides secure access between EC2 and Amazon S3.


GNU nano 8.3
deploy.sh
Modified
rm -rf /var/www/html/*
aws s3 sync s3://atharva242005 /var/www/html/
chown -R apache:apache /var/www/html
chmod -R 755 /var/www/html
systemctl restart httpd
echo ""
echo "=============================="
echo " Deployment Successful!"
echo "=============================="
^G Help
^X Exit
^O Write Out
^R Read File
^W Where Is
^\ Replace
^K Cut
^U Paste
^T Execute
^J Justify
^C Location
^/ Go To Line
M-U Undo
M-E Redo
M-A Set Mark
M-6 Copy
M-] To Bracket
^B Where Was
M-B Previous
M-F Next
i-0265b64f959e39f9a (Automated-Web_Server)
PublicIPs: 3.122.179.84 PrivateIPs: 172.31.43.227
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Free Cloud Computing Serv
Instances | EC2 | eu-central
EC2 Instance Connect | eu-c
Troubleshooting an error - C
AWS Automated Web Serve
Ask Gemini
eu-central-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=eu-central-1&connType=standard&instanceId=i-0265b64f959e39f9a&osUser=ec2-user&sshPort=22&addr...
aws
Search
[Alt+S]
Europe (Frankfurt)
atharva_aws (165044446808)
atharva_aws
[ec2-user@ip-172-31-43-227 ~]$ sudo ./deploy.sh
==========
AWS Web Server Deployment
==========
[1/5] Updating packages...
Last metadata expiration check: 0:27:07 ago on Fri Sep 4 14:05:35 2026.
Dependencies resolved.
Nothing to do.
Complete!
[2/5] Installing Apache...
Last metadata expiration check: 0:27:08 ago on Fri Sep 4 14:05:35 2026.
Package httpd-2.4.68-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[3/5] Starting Apache...
[4/5] Installing AWS CLI...
Last metadata expiration check: 0:27:09 ago on Fri Sep 4 14:05:35 2026.
Package awscli-2-2.33.15-1.amzn2023.0.1.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[5/5] Downloading website...
download: s3://atharva242005/style.css to ../../var/www/html/style.css
download: s3://atharva242005/script.js to ../../var/www/html/script.js
download: s3://atharva242005/index.html to ../../var/www/html/index.html
==========
Deployment Successful!
==========
[ec2-user@ip-172-31-43-227 ~]$
i-0265b64f959e39f9a (Automated-Web_Server)
PublicIPs: 3.122.179.84 PrivateIPs: 172.31.43.227
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
25°C
Mostly cloudy
Search
ENG
IN
20:02
04-09-2026